

AssanPay APIs — Signed Request Manual

Full-security integration with HMAC-SHA256 request signatures. Every request is authenticated with your API key plus a cryptographic signature derived from your API secret.

API Index

POST	/api/checkout/sessions	Checkout Session API
POST	/api/checkout/sessions	Checkout Session Encrypted API
POST	/api/payments	H2H Payin API
POST	/api/payments	H2H Payin Encrypted API
POST	/api/payments/p2p/submit	P2P TrxID Submit API
POST	/api/payments/p2p/submit	P2P TrxID Submit Encrypted API
GET	/api/merchant/status-inquiry?type=payin&orderId={{orderId}}	Payin Status Inquiry
POST	/api/merchant/wallet-payouts	H2H Payout API
POST	/api/merchant/wallet-payouts	H2H Payout Encrypted API
GET	/api/merchant/status-inquiry?type=payout&orderId={{orderId}}	Payout Status Inquiry
GET	/api/merchant/balance/h2h	Balance Check API
WEBHOOK	Callback Signature Verification	Callback Signature Verification
WEBHOOK	Payin Callback	Payin Callback
WEBHOOK	Payout Callback	Payout Callback

Overview

This manual guides you through implementing AssanPay's Host-to-Host (H2H) APIs with IP allowlist + API Secret authentication. Every sensitive endpoint requires HMAC-SHA256 request signatures in addition to the API key and IP check.

- branchCode is required in every request. Create branches from the Branches page, and each branch is approved by admin.
- Your server IPs must be whitelisted before H2H APIs can be used.
- API key and API secret are generated from the Settings page.
- Currency is BDT. Send amounts as plain integers (500) — a decimal point is refused even when whole.
- Every transaction belongs to one of two money streams, P2C or P2P, selected with the mode field.
- Final response status codes are listed in the Status Codes section at the end of this manual.
- For callback endpoints, return the documented 2xx response to acknowledge receipt and avoid retries.
- Request encryption can use the sub-merchant API secret, but response decryption and callback signature generation must use the main merchant API secret.

Request Signature

Cryptographic verification of every request

A signature is a cryptographic verification value used to confirm that the request was created by an authorized client and that its content has not been altered in transit.

Which API secret to use

Request encryption can be performed using the sub-merchant's API secret. However, response decryption and callback signature generation/verification must always use the main merchant's API secret.

Worked Example

INPUT	SAMPLE VALUE
Method	POST
Path with Query	/api/checkout/sessions
Timestamp	1712345678
Nonce	550e8400-e29b-41d4-a716-446655440000
Request Body	{"amount":500,"currency":"BDT"}
API Secret	your_api_secret

Signature Formula

Signature Formula

```
bodyHash = SHA256(requestBody)
canonical = METHOD + "\n" + PATH_WITH_QUERY + "\n" + TIMESTAMP + "\n" + NONCE + "\n"
+ BODY_HASH
signature = Base64(HMAC-SHA256(canonical, API_SECRET))
```

- PATH_WITH_QUERY includes the query string when present — GET requests sign their query too.
- BODY_HASH is lowercase hex. An empty body hashes the empty string.
- Clock skew allowed is ± 300 seconds, and the nonce is single-use per merchant.
- With a sub-merchant API key, sign with the sub-merchant's secret, otherwise the merchant's.

Request Headers

Request Headers

```
X-API-KEY: your_api_key
X-TIMESTAMP: 1712345678
X-NONCE: 550e8400-e29b-41d4-a716-446655440000
X-SIGNATURE: AbCdEf123456ExampleSignature==
Content-Type: application/json
```

Implementation Example

Implementation Example

```
function generateRequestSignature(
  { method, path, timestamp, nonce, rawBody },
  apiSecret
) {
  const bodyString = rawBody || '';

  // Step 1: Body hash (SHA256 hex)
  const bodyHash = crypto.createHash('sha256')
    .update(bodyString)
    .digest('hex');

  // Step 2: Canonical string
  const canonical = `${method}\n${path}\n${timestamp}\n${nonce}\n${bodyHash}`;

  // Step 3: Signature (HMAC SHA256 -> Base64)
  const signature = crypto.createHmac('sha256', apiSecret)
    .update(canonical)
    .digest('base64');

  return { signature, canonical, bodyHash };
}
```

Failure Responses

MISSING_AUTH_HEADERS

```
{
  "success": false,
  "errorCode": "MISSING_AUTH_HEADERS",
  "message": "Missing signature headers",
  "details": null,
  "timestamp": "2026-07-20T14:22:25+06:00"
}
```

INVALID_SIGNATURE

```
{
  "success": false,
  "errorCode": "INVALID_SIGNATURE",
  "message": "Invalid signature"
}
```

INVALID_NONCE

```
{
  "success": false,
  "errorCode": "INVALID_NONCE",
  "message": "Invalid nonce"
}
```

REQUEST_EXPIRED

```
{
  "success": false,
  "errorCode": "REQUEST_EXPIRED",
  "message": "Request expired"
}
```

Checkout Session API

Hosted checkout page for customer-initiated payments (P2C and P2P)

POST /api/checkout/sessions

Generates a unique checkout session URL per transaction. Share this URL with the customer to complete payment on the hosted checkout page.

Always redirect on checkoutUrl

Redirect the customer to checkoutUrl. If you build your own P2P screen instead, use walletNumber as the push-to number and then send the customer's TrxID to the P2P TrxID Submit API.

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds. Clock skew allowed is ± 300 seconds.
X-NONCE	Required	Unique random value generated for each request. Single-use per merchant.
X-SIGNATURE	Required	Generated request signature

Request Body Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
mode	string	Required	P2C or P2P (case-insensitive). Decides the money stream.
paymentMethodName	string	Required	Wallet brand — bkaash or nagad
amount	number	Required	Greater than 0, sent as a plain integer (500). A decimal point is refused even when the value is whole — 500.00 fails too.
branchCode	string	Required	Merchant branch code. Must match your sub-merchant key if branch-scoped.
customerEmail	string	Required	Valid format email address
currency	string	Optional	Defaults to BDT
orderId	string	Optional	6-32 chars, letters and digits, optional hyphens between segments. Auto-generated if omitted.
customerContact	string	Optional	If sent, must be 01XXXXXXXXXX (11 digits) or 880... (13 digits)
customerName	string	Optional	Customer name
callbackUrl	string	Optional	No webhook is sent without it
successUrl	string	Optional	Redirect URL after successful payment
failedUrl	string	Optional	Redirect URL after failed payment
pendingUrl	string	Optional	Redirect URL for pending

Request Body

Request Body

```
{
  "mode": "P2C",
  "paymentMethodName": "bkash",
  "amount": 500,
  "currency": "BDT",
  "orderId": "ORD-12345",
  "branchCode": "123456",
  "customerName": "Rahim Uddin",
  "customerEmail": "rahim@example.com",
  "customerContact": "01700000000",
  "callbackUrl": "https://merchant.com/webhooks/assanpay",
  "successUrl": "https://merchant.com/thank-you",
  "failedUrl": "https://merchant.com/failed",
  "pendingUrl": "https://merchant.com/pending"
}
```

Success Response — P2C

Success Response — P2C

```
{
  "sessionId": "cs_9f2c...",
  "orderId": "ORD-12345",
  "paymentId": 111127,
  "checkoutUrl": "https://payment.bkash.com/checkout/xxxx",
  "expiresAt": 1753000000,
  "status": "PENDING",
  "mode": "P2C",
  "walletNumber": null
}
```

Success Response — P2P

● ● ● Success Response — P2P

```
{
  "sessionId": "cs_71ab...",
  "orderId": "ORD-12346",
  "paymentId": 111128,
  "checkoutUrl": "https://business.assanpay.com/pay?sessionId=cs_71ab...",
  "expiresAt": 1753000900,
  "status": "PENDING",
  "mode": "P2P",
  "walletNumber": "01812345678"
}
```

Failure Responses

● ● ● INVALID_REQUEST

```
{
  "success": false,
  "errorCode": "INVALID_REQUEST",
  "message": "mode is required and must be P2C or P2P"
}
```

● ● ● INVALID_REQUEST

```
{
  "success": false,
  "errorCode": "INVALID_REQUEST",
  "message": "Amount must be a whole number (no decimals)"
}
```

● ● ● INVALID_BRANCH_CODE

```
{
  "success": false,
  "errorCode": "INVALID_BRANCH_CODE",
  "message": "branchCode does not match authenticated sub merchant"
}
```

PAYMENT_METHOD_NOT_ALLOWED

```
{
  "success": false,
  "errorCode": "PAYMENT_METHOD_NOT_ALLOWED",
  "message": "Payment method not enabled for merchant"
}
```

DUPLICATE_ORDER

```
{
  "success": false,
  "errorCode": "DUPLICATE_ORDER",
  "message": "Order ID already exists"
}
```

Checkout Session Encrypted API

AES-256-GCM encrypted checkout payload — signature required

POST /api/checkout/sessions

Signature on encrypted body

The signature must be generated using the final encrypted request body, not the original plaintext payload.

Which API secret to use

Request encryption can be performed using the sub-merchant's API secret. However, response decryption and callback signature generation/verification must always use the main merchant's API secret.

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds. Clock skew allowed is ± 300 seconds.
X-NONCE	Required	Unique random value generated for each request. Single-use per merchant.
X-SIGNATURE	Required	Generated request signature

Encrypted Body Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
encrypted	bool	Required	Must be true to indicate encrypted payload
alg	string	Required	Must be AES/GCM
iv	string	Required	Base64 URL-safe encoded IV (12 bytes)
cipherText	string	Required	Base64 URL-safe encoded encrypted payload + auth tag

How to Prepare an Encrypted Body

1

Create Original Payload

Build the standard JSON payload.

2

Derive Encryption Key

key = HMAC-SHA256('assanpay:req', API_SECRET)

3

Encrypt Payload

AES-256-GCM with random 12-byte IV. Append auth tag to ciphertext.

4

Encode Values

Base64 URL-safe encode the IV and (ciphertext + tag).

5

Build Final Body

Wrap in { encrypted, alg, iv, cipherText }.

Original Payload (before encryption)

Original Payload (before encryption)

```
{
  "mode": "P2C",
  "paymentMethodName": "bkash",
  "amount": 500,
  "currency": "BDT",
  "orderId": "ORD-12345",
  "branchCode": "123456",
  "customerName": "Rahim Uddin",
  "customerEmail": "rahim@example.com",
  "customerContact": "01700000000",
  "callbackUrl": "https://merchant.com/webhooks/assanpay",
  "successUrl": "https://merchant.com/thank-you",
  "failedUrl": "https://merchant.com/failed",
  "pendingUrl": "https://merchant.com/pending"
}
```

Final Encrypted Body

Final Encrypted Body

```
{
  "encrypted": true,
  "alg": "AES/GCM",
  "iv": "vXFGUThmJYdsNrkc",
  "cipherText": "9TdgmZv_Fvj_mecL..."
}
```

Encryption Example

Encryption Example

```
function encryptPayload(payload, apiSecret) {
  const b64url = (buf) =>
    buf.toString('base64')
      .replace(/\+/g, '-')
      .replace(/\//g, '_')
      .replace(/=+$/g, '');

  const key = crypto
    .createHmac('sha256', apiSecret)
    .update('assanpay:req')
    .digest();

  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);
  const plain = Buffer.from(JSON.stringify(payload), 'utf8');
  const ciphertext = Buffer.concat([cipher.update(plain), cipher.final()]);
  const tag = cipher.getAuthTag();
  const cipherAll = Buffer.concat([ciphertext, tag]);

  return {
    encrypted: true,
    alg: 'AES/GCM',
    iv: b64url(iv),
    cipherText: b64url(cipherAll)
  };
}
```

Success Response

Success Response

```
{
  "sessionId": "cs_71ab...",
  "orderId": "ORD-12346",
  "paymentId": 111128,
  "checkoutUrl": "https://business.assanpay.com/pay?sessionId=cs_71ab...",
  "expiresAt": 1753000900,
  "status": "PENDING",
  "mode": "P2P",
  "walletNumber": "01812345678"
}
```

Decryption Example

Decryption Example

```
function decryptResponse(encryptedBody, apiSecret) {
  const b64urlToBuffer = (str) => {
    str = str.replace(/-/g, '+').replace(/_/g, '/');
    while (str.length % 4) str += '=';
    return Buffer.from(str, 'base64');
  };

  const key = crypto
    .createHmac('sha256', apiSecret)
    .update('assanpay:res')
    .digest();

  const iv = b64urlToBuffer(encryptedBody.iv);
  const cipherData = b64urlToBuffer(encryptedBody.cipherText);
  const tag = cipherData.slice(cipherData.length - 16);
  const ciphertext = cipherData.slice(0, cipherData.length - 16);

  const decipher = crypto.createDecipheriv('aes-256-gcm', key, iv);
  decipher.setAuthTag(tag);
  const decrypted = Buffer.concat([decipher.update(ciphertext), decipher.final()]);

  return JSON.parse(decrypted.toString('utf8'));
}
```

H2H Payin API

Immediate pending, final status via callback

POST /api/payments

On P2C, only bkaash is open on this API

Any other P2C method is rejected with PAYMENT_METHOD_NOT_ALLOWED — use the Checkout Session API for it. On bkaash P2C, customerContact is required and must be the customer's real wallet number: the payment is matched and verified against it, so a wrong number cannot be collected.

walletNumber is the push-to number

The customer pays by pushing the amount to walletNumber, then you send their TrxID to the P2P TrxID Submit API. walletNumber carries a value only on a PENDING P2P.

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds. Clock skew allowed is ± 300 seconds.
X-NONCE	Required	Unique random value generated for each request. Single-use per merchant.
X-SIGNATURE	Required	Generated request signature

Request Body Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
paymentMethodName	string	Required	Wallet brand — bkaash or nagad
branchCode	string	Required	Merchant branch code. Must match your sub-merchant key.
amount	number	Required	Greater than 0, sent as a plain integer (500). A decimal point is refused even when the value is whole — 500.00 fails too.
mode	string	Optional	P2P, or P2C with bkaash. Any other P2C method is rejected here — use the Checkout Session API for it.
orderId	string	Optional	6-32 chars, letters and digits, optional hyphens between segments
currency	string	Optional	3-letter ISO code. Defaults to BDT.
customerContact	string	Optional	Valid BD wallet number. Required on bkaash P2C, and must be the customer's real number — the payment is verified against it.
customerName	string	Optional	Auto-filled if blank
customerEmail	string	Optional	Auto-filled if blank
callbackUrl	string	Optional	Merchant callback URL for status updates

Request Body

Request Body

```
{
  "paymentMethodName": "bkaash",
  "mode": "P2P",
  "amount": 500,
  "currency": "BDT",
  "orderId": "ORD-12345",
  "branchCode": "123456",
  "customerContact": "01700000000",
  "customerName": "Rahim Uddin",
  "customerEmail": "rahim@example.com",
  "callbackUrl": "https://merchant.com/webhooks/assanpay"
}
```

Immediate Response (Pending)

● ● ● Immediate Response (Pending)

```
{
  "success": false,
  "status": "PENDING",
  "statusCode": "202",
  "message": "Push the amount to the wallet number, then submit your TrxID",
  "type": "payin",
  "mode": "P2P",
  "orderId": "ORD-12345",
  "paymentId": "PMT-8f21...",
  "amount": 500.00,
  "receivedAmount": null,
  "currency": "BDT",
  "walletNumber": "01812345678",
  "transactionId": null,
  "reference": null,
  "accountNumber": "017000000000",
  "accountTitle": null,
  "rail": "MW",
  "redirectUrl": null,
  "qrPayload": null,
  "reason": "PENDING",
  "occurredAt": "2026-07-20T14:22:25+06:00"
}
```

Failed Response

Failed Response

```
{
  "success": false,
  "status": "FAILED",
  "statusCode": "400",
  "message": "Payment initiation failed.",
  "type": "payin",
  "mode": "P2P",
  "orderId": "ORD-12345",
  "paymentId": "PMT-8f21...",
  "amount": 500.00,
  "receivedAmount": null,
  "currency": "BDT",
  "walletNumber": null,
  "transactionId": null,
  "reference": null,
  "accountNumber": "017000000000",
  "accountTitle": null,
  "rail": "MW",
  "redirectUrl": null,
  "qrPayload": null,
  "reason": "FAILED",
  "occurredAt": "2026-07-20T14:22:25+06:00"
}
```

Failure Responses

MISSING_AUTH_HEADERS

```
{
  "success": false,
  "errorCode": "MISSING_AUTH_HEADERS",
  "message": "Missing signature headers"
}
```

INVALID_SIGNATURE

```
{
  "success": false,
  "errorCode": "INVALID_SIGNATURE",
  "message": "Invalid signature"
}
```

PAYMENT_METHOD_NOT_ALLOWED

```
{
  "success": false,
  "errorCode": "PAYMENT_METHOD_NOT_ALLOWED",
  "message": "P2C payments are supported via checkout only"
}
```

INVALID_REQUEST

```
{
  "success": false,
  "errorCode": "INVALID_REQUEST",
  "message": "Amount must be a whole number (no decimals)"
}
```

INVALID_REQUEST – bKash P2C

```
{
  "success": false,
  "errorCode": "INVALID_REQUEST",
  "message": "Enter a valid bKash number (01XXXXXXXXXX)"
}
```

DUPLICATE_ORDER

```
{
  "success": false,
  "errorCode": "DUPLICATE_ORDER",
  "message": "Order ID already exists"
}
```

H2H Payin Encrypted API

AES-256-GCM encrypted payload transmission

POST /api/payments

Signature on encrypted body

The signature must be generated using the final encrypted request body, not the original plaintext payload.

Which API secret to use

Request encryption can be performed using the sub-merchant's API secret. However, response decryption and callback signature generation/verification must always use the main merchant's API secret.

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds. Clock skew allowed is ± 300 seconds.
X-NONCE	Required	Unique random value generated for each request. Single-use per merchant.
X-SIGNATURE	Required	Generated request signature

Encrypted Body Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
encrypted	bool	Required	Must be true to indicate encrypted payload
alg	string	Required	Must be AES/GCM
iv	string	Required	Base64 URL-safe encoded IV (12 bytes)
cipherText	string	Required	Base64 URL-safe encoded encrypted payload + auth tag

How to Prepare an Encrypted Body

1

Create Original Payload

Build the standard JSON payload.

2

Derive Encryption Key

key = HMAC-SHA256('assanpay:req', API_SECRET)

3

Encrypt Payload

AES-256-GCM with random 12-byte IV. Append auth tag to ciphertext.

4

Encode Values

Base64 URL-safe encode the IV and (ciphertext + tag).

5

Build Final Body

Wrap in { encrypted, alg, iv, cipherText }.

Original Payload (before encryption)

Original Payload (before encryption)

```
{
  "paymentMethodName": "bkash",
  "mode": "P2P",
  "amount": 500,
  "currency": "BDT",
  "orderId": "ORD-12345",
  "branchCode": "123456",
  "customerContact": "017000000000",
  "customerName": "Rahim Uddin",
  "customerEmail": "rahim@example.com",
  "callbackUrl": "https://merchant.com/webhooks/assanpay"
}
```

Final Encrypted Body

Final Encrypted Body

```
{
  "encrypted": true,
  "alg": "AES/GCM",
  "iv": "vXFGUThmJYdsNrkc",
  "cipherText": "9TdgmZv_Fvj_mecL..."
}
```

Encryption Example

Encryption Example

```
function encryptPayload(payload, apiSecret) {
  const b64url = (buf) =>
    buf.toString('base64')
      .replace(/\+/g, '-')
      .replace(/\//g, '_')
      .replace(/=+$/g, '');

  const key = crypto
    .createHmac('sha256', apiSecret)
    .update('assanpay:req')
    .digest();

  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);
  const plain = Buffer.from(JSON.stringify(payload), 'utf8');
  const ciphertext = Buffer.concat([cipher.update(plain), cipher.final()]);
  const tag = cipher.getAuthTag();
  const cipherAll = Buffer.concat([ciphertext, tag]);

  return {
    encrypted: true,
    alg: 'AES/GCM',
    iv: b64url(iv),
    cipherText: b64url(cipherAll)
  };
}
```

Success Response (Encrypted)

● ● ● Success Response (Encrypted)

```
{
  "encrypted": true,
  "alg": "AES/GCM",
  "iv": "NIIdWfHS6pNe9ow7e",
  "cipherText": "Wquopg9kMpJpkCbGi3Li72EwMBL..."
}
```

Decryption Example

● ● ● Decryption Example

```
function decryptResponse(encryptedBody, apiSecret) {
  const b64urlToBuffer = (str) => {
    str = str.replace(/-/g, '+').replace(/_/g, '/');
    while (str.length % 4) str += '=';
    return Buffer.from(str, 'base64');
  };

  const key = crypto
    .createHmac('sha256', apiSecret)
    .update('assanpay:res')
    .digest();

  const iv = b64urlToBuffer(encryptedBody.iv);
  const cipherData = b64urlToBuffer(encryptedBody.cipherText);
  const tag = cipherData.slice(cipherData.length - 16);
  const ciphertext = cipherData.slice(0, cipherData.length - 16);

  const decipher = crypto.createDecipheriv('aes-256-gcm', key, iv);
  decipher.setAuthTag(tag);
  const decrypted = Buffer.concat([decipher.update(ciphertext), decipher.final()]);

  return JSON.parse(decrypted.toString('utf8'));
}
```

Failure Responses

Encrypted Response (Example)

```
{
  "encrypted": true,
  "alg": "AES/GCM",
  "iv": "NIIdWfHS6pNe9ow7e",
  "cipherText": "Wquopg9kMpJpkCbGi3Li72EwMBL..."
}
```

MISSING_AUTH_HEADERS

```
{
  "success": false,
  "errorCode": "MISSING_AUTH_HEADERS",
  "message": "Missing signature headers",
  "details": null,
  "timestamp": "2026-07-20T14:22:25+06:00"
}
```

P2P TrxID Submit API

Submit the customer's transaction reference after they push the money

POST /api/payments/p2p/submit

After the customer pushes the amount to walletNumber, submit the TrxID they received. Headers are the same as your payin request (signed mode: signature headers).

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds. Clock skew allowed is ± 300 seconds.
X-NONCE	Required	Unique random value generated for each request. Single-use per merchant.
X-SIGNATURE	Required	Generated request signature

Request Body Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
orderId	string	Required	Order ID used in the original payin request
trxId	string	Required	Transaction reference the customer received from the wallet

Request Body

Request Body

```
{
  "orderId": "ORD-12345",
  "trxId": "75C0J0K0"
}
```

Accepted Response

Accepted Response

```
{
  "orderId": "ORD-12345",
  "paymentId": 111127,
  "submitStatus": "SUBMITTED",
  "message": "Trade number submitted",
  "paymentStatus": "PENDING"
}
```

SUBMITTED is not success

paymentStatus stays PENDING until the wallet confirms. The final verdict arrives in the payin callback — never mark an order paid on this response.

Refused Response (same 200 OK shape)

● ● ● Refused Response (same 200 OK shape)

```
{
  "orderId": "ORD-12345",
  "paymentId": 111127,
  "submitStatus": "REJECTED",
  "message": "This transaction reference has already been used for another payment",
  "paymentStatus": "PENDING"
}
```

Read submitStatus, not the HTTP code

A refused submit is not an error envelope — it is a 200 OK with submitStatus REJECTED and the reason in message. The payment is still PENDING and the trade number is not stored, so the customer can correct it and submit again. Never re-initiate the order.

Failure Responses

● ● ● INVALID_REQUEST

```
{
  "success": false,
  "errorCode": "INVALID_REQUEST",
  "message": "orderId is required"
}
```

● ● ● RESOURCE_NOT_FOUND

```
{
  "success": false,
  "errorCode": "RESOURCE_NOT_FOUND",
  "message": "Payment not found"
}
```

P2P TrxID Submit Encrypted API

AES-256-GCM encrypted submit payload — signature required

POST /api/payments/p2p/submit

Signature required for encrypted submit

If request encryption is enabled for you, send the AES-256-GCM envelope instead of the plaintext body — sending plaintext then returns 400. The signature must be generated over the final encrypted body, so full signature headers are required even in IP whitelist mode. We decrypt to { orderId, trxId }; if response encryption is enabled, the response comes back as an encrypted envelope too.

Which API secret to use

Request encryption can be performed using the sub-merchant's API secret. However, response decryption and callback signature generation/verification must always use the main merchant's API secret.

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds. Clock skew allowed is ± 300 seconds.
X-NONCE	Required	Unique random value generated for each request. Single-use per merchant.
X-SIGNATURE	Required	Generated request signature

Encrypted Body Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
encrypted	bool	Required	Must be true to indicate encrypted payload
alg	string	Required	Must be AES/GCM
iv	string	Required	Base64 URL-safe encoded IV (12 bytes)
cipherText	string	Required	Base64 URL-safe encoded encrypted payload + auth tag

How to Prepare an Encrypted Body

1

Create Original Payload

Build the standard JSON payload.

2

Derive Encryption Key

key = HMAC-SHA256('assanpay:req', API_SECRET)

3

Encrypt Payload

AES-256-GCM with random 12-byte IV. Append auth tag to ciphertext.

4

Encode Values

Base64 URL-safe encode the IV and (ciphertext + tag).

5

Build Final Body

Wrap in { encrypted, alg, iv, cipherText }.

Original Payload (before encryption)

Original Payload (before encryption)

```
{
  "orderId": "ORD-12345",
  "trxId": "75C0J0K0"
}
```

Final Encrypted Body

Final Encrypted Body

```
{
  "encrypted": true,
  "alg": "AES/GCM",
  "iv": "vXFGUThmJYdsNrkc",
  "cipherText": "9TdgmZv_Fvj_mecL..."
}
```

Encryption Example

Encryption Example

```
function encryptPayload(payload, apiSecret) {
  const b64url = (buf) =>
    buf.toString('base64')
      .replace(/\+/g, '-')
      .replace(/\//g, '_')
      .replace(/=+$/g, '');

  const key = crypto
    .createHmac('sha256', apiSecret)
    .update('assanpay:req')
    .digest();

  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);
  const plain = Buffer.from(JSON.stringify(payload), 'utf8');
  const ciphertext = Buffer.concat([cipher.update(plain), cipher.final()]);
  const tag = cipher.getAuthTag();
  const cipherAll = Buffer.concat([ciphertext, tag]);

  return {
    encrypted: true,
    alg: 'AES/GCM',
    iv: b64url(iv),
    cipherText: b64url(cipherAll)
  };
}
```

Success Response (Encrypted)

Success Response (Encrypted)

```
{
  "encrypted": true,
  "alg": "AES/GCM",
  "iv": "NIdWfHS6pNe9ow7e",
  "cipherText": "Wquopg9kMpJpkCbGi3Li72EwMBL..."
}
```

Decryption Example

Decryption Example

```
function decryptResponse(encryptedBody, apiSecret) {
  const b64urlToBuffer = (str) => {
    str = str.replace(/-/g, '+').replace(/_/g, '/');
    while (str.length % 4) str += '=';
    return Buffer.from(str, 'base64');
  };

  const key = crypto
    .createHmac('sha256', apiSecret)
    .update('assanpay:res')
    .digest();

  const iv = b64urlToBuffer(encryptedBody.iv);
  const cipherData = b64urlToBuffer(encryptedBody.cipherText);
  const tag = cipherData.slice(cipherData.length - 16);
  const ciphertext = cipherData.slice(0, cipherData.length - 16);

  const decipher = crypto.createDecipheriv('aes-256-gcm', key, iv);
  decipher.setAuthTag(tag);
  const decrypted = Buffer.concat([decipher.update(ciphertext), decipher.final()]);

  return JSON.parse(decrypted.toString('utf8'));
}
```

Payin Status Inquiry

Query the current state of a pay-in by order ID

GET /api/merchant/status-inquiry?type=payin&orderId={{orderId}}

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client

Query Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
orderId	string	Required	Your order id, exactly as sent at initiate
type	string	Required	payin or payout — picks which stream to read

Signing is skipped on status-inquiry paths

Status-inquiry endpoints are exempt from the request signature, so only the API key is needed. If you do sign them, remember the query string is part of the signature payload.

Success Response

Success Response

```
{
  "success": true,
  "status": "SUCCESS",
  "statusCode": "200",
  "message": "Successful",
  "type": "payin",
  "mode": "P2P",
  "orderId": "ORD-12345",
  "paymentId": "T20260723145843800mr",
  "amount": 500.00,
  "receivedAmount": 450.00,
  "currency": "BDT",
  "walletNumber": "01632036725",
  "transactionId": "75PTKI7N",
  "reference": "75PTKI7N",
  "accountNumber": "017000000000",
  "accountTitle": null,
  "rail": "MW",
  "redirectUrl": null,
  "qrPayload": null,
  "reason": "Successful",
  "occurredAt": "2026-07-23T20:58:43+06:00",
  "successAt": "2026-07-23T21:04:04+06:00"
}
```

Pending Response

Pending Response

```
{
  "success": false,
  "status": "PENDING",
  "statusCode": "202",
  "message": "Push the amount to the wallet number, then submit your TrxID",
  "type": "payin",
  "mode": "P2P",
  "orderId": "ORD-12345",
  "paymentId": "T20260723145843800mr",
  "amount": 500.00,
  "receivedAmount": null,
  "currency": "BDT",
  "walletNumber": "01632036725",
  "transactionId": null,
  "reference": null,
  "accountNumber": "017000000000",
  "accountTitle": null,
  "rail": "MW",
  "redirectUrl": null,
  "qrPayload": null,
  "reason": "PENDING",
  "occurredAt": "2026-07-23T20:58:43+06:00",
  "time": "2026-07-23T21:01:10+06:00"
}
```

Failed Response

Failed Response

```
{
  "success": false,
  "status": "FAILED",
  "statusCode": "400",
  "message": "Payment could not be initiated, please try again",
  "type": "payin",
  "mode": "P2P",
  "orderId": "ORD-12345",
  "paymentId": "T20260723145843800mr",
  "amount": 500.00,
  "receivedAmount": null,
  "currency": "BDT",
  "walletNumber": null,
  "transactionId": null,
  "reference": null,
  "accountNumber": "017000000000",
  "accountTitle": null,
  "rail": "MW",
  "redirectUrl": null,
  "qrPayload": null,
  "reason": "Payment could not be initiated, please try again",
  "occurredAt": "2026-07-23T20:58:43+06:00",
  "failedAt": "2026-07-23T20:59:02+06:00"
}
```

Settle on receivedAmount, not on amount

amount is what you requested at initiate; receivedAmount is what actually arrived. On P2P the customer pushes manually and may send more or less — it still completes as SUCCESS, and settlement and commission run on what actually arrived. receivedAmount is always present, but stays null until a SUCCESS captures it.

Failure Responses

   MERCHANT_NOT_FOUND

```
{
  "success": false,
  "errorCode": "MERCHANT_NOT_FOUND",
  "message": "Merchant not authenticated"
}
```

   INVALID_REQUEST

```
{
  "success": false,
  "errorCode": "INVALID_REQUEST",
  "message": "Order id is required"
}
```

   RESOURCE_NOT_FOUND

```
{
  "success": false,
  "errorCode": "RESOURCE_NOT_FOUND",
  "message": "Payment not found"
}
```

H2H Payout API

Disburse to a BD wallet from your system

POST /api/merchant/wallet-payouts

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds. Clock skew allowed is ± 300 seconds.
X-NONCE	Required	Unique random value generated for each request. Single-use per merchant.
X-SIGNATURE	Required	Generated request signature

Request Body Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
method	string	Required	Bkash or Nagad. Send the plain brand — the stream suffix is added server-side. Aliases: payment_method, brand.
mode	string	Required	P2C or P2P. Picks which balance pool is debited and which payout method row applies. No default.
amount	number	Required	Greater than 0, sent as a plain integer (1000). A decimal point is refused even when the value is whole, and is never rounded.
accountNo	string	Required	Destination wallet number (01XXXXXXXXXX)
branchCode	string	Required	Merchant branch code. Alias branch_code.
orderId	string	Optional	Unique payout order ID. Alias order_id. Auto-generated if omitted.
callbackUrl	string	Optional	Alias callback_url. No payout webhook is sent without it.
beneficiaryName	string	Optional	Beneficiary name, passed through
purpose	string	Optional	Purpose or description of the payout
phone	string	Optional	Beneficiary phone number, if applicable

mode is required

mode selects the stream the payout belongs to. The available balance is checked against that mode's pool, and the amount is debited from that same pool — so a payout sent with mode P2P is validated and cut from the P2P balance, never the combined total.

Request Body

Request Body

```
{
  "method": "Bkash",
  "mode": "P2P",
  "amount": 1000,
  "accountNo": "01711111111",
  "branchCode": "123456",
  "orderId": "PO-98765",
  "beneficiaryName": "Karim",
  "callbackUrl": "https://merchant.com/webhooks/assanpay-payout"
}
```

Pending Response

Pending Response

```
{
  "success": false,
  "status": "PENDING",
  "statusCode": "202",
  "message": "Payout request accepted",
  "type": "payout",
  "mode": "P2P",
  "orderId": "PO-98765",
  "payoutId": 4412,
  "amount": 1000.00,
  "currency": "BDT",
  "transactionId": "out_111",
  "reference": "out_111",
  "accountNumber": "01711111111",
  "accountTitle": "Karim",
  "rail": "MW",
  "reason": "PENDING",
  "occurredAt": "2026-07-20T14:30:00+06:00"
}
```

Failure Responses

INVALID_REQUEST

```
{
  "success": false,
  "errorCode": "INVALID_REQUEST",
  "message": "mode is required and must be P2C or P2P"
}
```

PAYMENT_METHOD_NOT_ALLOWED

```
{
  "success": false,
  "errorCode": "PAYMENT_METHOD_NOT_ALLOWED",
  "message": "Payout method Bkash P2P belongs to the P2P stream but mode is P2C"
}
```

INVALID_REQUEST

```
{
  "success": false,
  "errorCode": "INVALID_REQUEST",
  "message": "Payout amount must be a whole number (no decimals)"
}
```

OPERATION_NOT_ALLOWED

```
{
  "success": false,
  "errorCode": "OPERATION_NOT_ALLOWED",
  "message": "Insufficient merchant balance for payout"
}
```

DUPLICATE_ORDER

```
{
  "success": false,
  "errorCode": "DUPLICATE_ORDER",
  "message": "Order ID already exists"
}
```

H2H Payout Encrypted API

Encrypted payout with AES-256-GCM

POST /api/merchant/wallet-payouts

Signature on encrypted body

Generate the signature using the final encrypted request body, not the original plaintext payload.

Which API secret to use

Request encryption can be performed using the sub-merchant's API secret. However, response decryption and callback signature generation/verification must always use the main merchant's API secret.

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds. Clock skew allowed is ± 300 seconds.
X-NONCE	Required	Unique random value generated for each request. Single-use per merchant.
X-SIGNATURE	Required	Generated request signature

Encrypted Body Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
encrypted	bool	Required	Must be true to indicate encrypted payload
alg	string	Required	Must be AES/GCM
iv	string	Required	Base64 URL-safe encoded IV (12 bytes)
cipherText	string	Required	Base64 URL-safe encoded encrypted payload + auth tag

Original Payload (before encryption)

Original Payload (before encryption)

```
{
  "method": "Bkash",
  "mode": "P2P",
  "amount": 1000,
  "accountNo": "01711111111",
  "branchCode": "123456",
  "orderId": "PO-98765",
  "beneficiaryName": "Karim",
  "callbackUrl": "https://merchant.com/webhooks/assanpay-payout"
}
```

Final Encrypted Body

Final Encrypted Body

```
{
  "encrypted": true,
  "alg": "AES/GCM",
  "iv": "vXFGUThmJYdsNrkc",
  "cipherText": "9TdgmZv_Fvj_mecL..."
}
```

Encryption Example

Encryption Example

```
function encryptPayload(payload, apiSecret) {
  const b64url = (buf) =>
    buf.toString('base64')
      .replace(/\+/g, '-')
      .replace(/\//g, '_')
      .replace(/=+$/g, '');

  const key = crypto
    .createHmac('sha256', apiSecret)
    .update('assanpay:req')
    .digest();

  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);
  const plain = Buffer.from(JSON.stringify(payload), 'utf8');
  const ciphertext = Buffer.concat([cipher.update(plain), cipher.final()]);
  const tag = cipher.getAuthTag();
  const cipherAll = Buffer.concat([ciphertext, tag]);

  return {
    encrypted: true,
    alg: 'AES/GCM',
    iv: b64url(iv),
    cipherText: b64url(cipherAll)
  };
}
```

Decryption Example

Decryption Example

```
function decryptResponse(encryptedBody, apiSecret) {
  const b64urlToBuffer = (str) => {
    str = str.replace(/-/g, '+').replace(/_/g, '/');
    while (str.length % 4) str += '=';
    return Buffer.from(str, 'base64');
  };

  const key = crypto
    .createHmac('sha256', apiSecret)
    .update('assanpay:res')
    .digest();

  const iv = b64urlToBuffer(encryptedBody.iv);
  const cipherData = b64urlToBuffer(encryptedBody.cipherText);
  const tag = cipherData.slice(cipherData.length - 16);
  const ciphertext = cipherData.slice(0, cipherData.length - 16);

  const decipher = crypto.createDecipheriv('aes-256-gcm', key, iv);
  decipher.setAuthTag(tag);
  const decrypted = Buffer.concat([decipher.update(ciphertext), decipher.final()]);

  return JSON.parse(decrypted.toString('utf8'));
}
```

Payout Status Inquiry

Query the current state of a payout by order ID

GET /api/merchant/status-inquiry?type=payout&orderId={{orderId}}

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client

Query Parameters

FIELD	TYPE	REQUIRED	DESCRIPTION
orderId	string	Required	Your order id, exactly as sent at initiate
type	string	Required	payin or payout — picks which stream to read

Success Response

Success Response

```
{
  "success": false,
  "status": "PENDING",
  "statusCode": "202",
  "message": "Payout request accepted",
  "type": "payout",
  "mode": "P2C",
  "orderId": "WP100000911115",
  "payoutId": 911115,
  "amount": 450.00,
  "currency": "BDT",
  "transactionId": "out_af7410b97620498cbfa95dd43c591115",
  "reference": "out_af7410b97620498cbfa95dd43c591115",
  "accountNumber": "01816937464",
  "accountTitle": null,
  "rail": "MW",
  "reason": "Payout request accepted",
  "occurredAt": "2026-07-23T21:15:59+06:00",
  "time": "2026-07-23T21:20:04+06:00"
}
```

Same timestamp rule as pay-in: PENDING gives time, COMPLETED gives successAt, FAILED gives failedAt.

Failure Responses

INVALID_REQUEST

```
{
  "success": false,
  "errorCode": "INVALID_REQUEST",
  "message": "Order id is required"
}
```

RESOURCE_NOT_FOUND

```
{
  "success": false,
  "errorCode": "RESOURCE_NOT_FOUND",
  "message": "Payout not found"
}
```

Balance Check API

Check merchant balance from your system

GET /api/merchant/balance/h2h

Required Headers

HEADER	REQUIRED	DESCRIPTION
X-API-KEY	Required	API key assigned to the client
X-TIMESTAMP	Required	Current Unix timestamp in seconds. Clock skew allowed is ± 300 seconds.
X-NONCE	Required	Unique random value generated for each request. Single-use per merchant.
X-SIGNATURE	Required	Generated request signature

Query params, all optional: mode, branchCode, subMerchantId. The query string is part of the signature payload, so sign the full path including it.

Success Response

Success Response

```
{
  "balanceAmount": 15000.00,
  "availableBalance": 14000.00,
  "pendingBalance": 1000.00,
  "currency": "BDT",
  "updatedAt": "2026-07-20T10:00:00",
  "payoutFreeze": false,
  "pools": [
    { "mode": "P2C", "balanceAmount": 10000.00, "availableBalance": 9500.00,
      "pendingBalance": 500.00 },
    { "mode": "P2P", "balanceAmount": 5000.00, "availableBalance": 4500.00,
      "pendingBalance": 500.00 }
  ]
}
```

Top-level balance is always combined

balanceAmount stays the combined total even when mode is sent. To show a single stream, read pools[0]. An unrecognised mode such as p2pp is not an error, it resolves to P2C — send the exact value.

Callback Signature Verification

Verify that webhooks are genuinely from AssanPay

A callback signature is used to verify that the webhook request was sent by an authorized source and that its content has not been tampered with. When a webhook is received, generate a signature from the received values and compare it with the signature sent in the request headers.

- Event ID: sent in header X-Assanpay-Event-Id (unique identifier for the event).
- Timestamp: sent in header X-Assanpay-Timestamp (Unix timestamp in seconds).
- Raw request body: the exact raw body string received in the webhook (do not parse or modify before hashing).
- API Secret: your shared secret key used to verify the signature.

Important: do not modify the body

Use the exact raw request body as received. Do not parse JSON, re-stringify, pretty-print, or change whitespace/newlines before generating the signature.

Which API secret to use

Request encryption can be performed using the sub-merchant's API secret. However, response decryption and callback signature generation/verification must always use the main merchant's API secret.

Verification Flow

1

Extract required headers

Read X-Assanpay-Event-Id, X-Assanpay-Timestamp, and X-Assanpay-Signature from the incoming webhook headers.

2

Build the canonical string

Combine EVENT_ID, TIMESTAMP, and RAW_BODY in this exact order, separated by a newline (\n).

3

Generate HMAC

Compute Base64(HMAC-SHA256(canonical, API_SECRET)).

4

Compare signatures

If generated signature matches the received X-Assanpay-Signature, treat the webhook as valid.

Signature Formula

Signature Formula

```
canonical = EVENT_ID + "\n" + TIMESTAMP + "\n" + RAW_BODY
signature = Base64(HMAC-SHA256(canonical, API_SECRET))
```

Verification Example

Verification Example

```
function generateCallbackSignature(rawBody, headers, apiSecret) {
  if (!apiSecret) throw new Error('Missing API secret');

  const eventId = headers['X-Assanpay-Event-Id'] || '';
  const timestamp = headers['X-Assanpay-Timestamp'] || '';
  const body = rawBody ? String(rawBody) : '';

  const canonical = `${eventId}\n${timestamp}\n${body}`;

  const signature = crypto
    .createHmac('sha256', apiSecret)
    .update(canonical)
    .digest('base64');

  return { canonical, signature };
}

function verifyCallbackSignature(rawBody, headers, apiSecret) {
  const generated = generateCallbackSignature(rawBody, headers, apiSecret);
  const receivedSignature = headers['X-Assanpay-Signature'] || '';
  return generated.signature === receivedSignature;
}
```

Payin Callback

Received webhook for payin status updates. Return any 2xx response to acknowledge receipt.

P2P amounts can differ — settle on receivedAmount

The customer pushes manually, so over or under payment still lands as SUCCESS. Settle against receivedAmount, not amount. For P2C they are equal. On P2P, receivedAmount only carries a value on SUCCESS — it is null on a FAILED callback, so a failure can never look collected.

AssanPay expects any 2xx response against the callback. If a 2xx response is not received, the callback is retried up to 12 times within a 120-second budget using exponential backoff. Only terminal statuses (SUCCESS and FAILED) are sent — never PENDING. Dedupe on X-Assanpay-Event-Id.

Headers

HEADER	REQUIRED	DESCRIPTION
X-Assanpay-Event-Id	Required	Unique event id. Dedupe on this value.
X-Assanpay-Timestamp	Required	Current Unix timestamp in seconds
X-Assanpay-Signature	Required	Generated request signature
X-Assanpay-Delivery-Seq	Optional	Present only on replays

Which API secret to use

Request encryption can be performed using the sub-merchant's API secret. However, response decryption and callback signature generation/verification must always use the main merchant's API secret.

Callback Body (Success JSON — P2C)

Callback Body (Success JSON — P2C)

```
{
  "success": true,
  "status": "SUCCESS",
  "statusCode": "200",
  "message": "SUCCESS",
  "type": "payin",
  "mode": "P2C",
  "orderId": "TS0676gb24",
  "paymentId": "T202608200616341g",
  "amount": 129,
  "receivedAmount": 129,
  "currency": "BDT",
  "transactionId": "DHK0NJBKUE",
  "reference": "DHK0NJBKUE",
  "accountNumber": "01816937464",
  "accountTitle": null,
  "walletNumber": "01828958921",
  "qrPayload": null,
  "redirectUrl": null,
  "rail": "MW",
  "reason": null,
  "occurredAt": "2026-08-20T12:16:34.152839+06:00",
  "successAt": "2026-08-20T12:20:56.189782672+06:00"
}
```

Callback Body (Success JSON — P2P)

Callback Body (Success JSON — P2P)

```
{
  "success": true,
  "status": "SUCCESS",
  "statusCode": "200",
  "message": "Payment successful",
  "type": "payin",
  "mode": "P2P",
  "orderId": "ORD-12345",
  "paymentId": "PMT-8f21...",
  "payoutId": null,
  "amount": 500.00,
  "receivedAmount": 500.00,
  "currency": "BDT",
  "transactionId": "75C0J0K0",
  "reference": "75C0J0K0",
  "accountNumber": "017000000000",
  "accountTitle": null,
  "walletNumber": null,
  "qrPayload": null,
  "rail": "MW",
  "reason": "SUCCESS",
  "successAt": "2026-07-20T14:25:10+06:00"
}
```

Callback Body (Failed JSON — P2P)

Callback Body (Failed JSON — P2P)

```
{
  "success": false,
  "status": "FAILED",
  "statusCode": "400",
  "message": "Payment failed",
  "type": "payin",
  "mode": "P2P",
  "orderId": "ORD-12345",
  "paymentId": "PMT-8f21...",
  "payoutId": null,
  "amount": 500.00,
  "receivedAmount": null,
  "currency": "BDT",
  "transactionId": "75C0J0K0",
  "reference": "75C0J0K0",
  "accountNumber": "017000000000",
  "accountTitle": null,
  "walletNumber": null,
  "qrPayload": null,
  "rail": "MW",
  "reason": "FAILED"
}
```

Callback Body (Encrypted)

Callback Body (Encrypted)

```
{
  "encrypted": true,
  "alg": "AES/GCM",
  "iv": "NIIdWfHS6pNe9ow7e",
  "cipherText": "Wquopg9kMpJpkCbGi3Li72EwMBL..."
}
```

Payout Callback

Received webhook for payout status updates. Return any 2xx response to acknowledge receipt.

AssanPay expects any 2xx response against the callback. If a 2xx response is not received, the callback is retried up to 12 times within a 120-second budget using exponential backoff. Only terminal statuses (SUCCESS and FAILED) are sent — never PENDING. Dedupe on X-Assanpay-Event-Id.

Headers

HEADER	REQUIRED	DESCRIPTION
X-Assanpay-Event-Id	Required	Unique event id. Dedupe on this value.
X-Assanpay-Timestamp	Required	Current Unix timestamp in seconds
X-Assanpay-Signature	Required	Generated request signature
X-Assanpay-Delivery-Seq	Optional	Present only on replays

Which API secret to use

Request encryption can be performed using the sub-merchant's API secret. However, response decryption and callback signature generation/verification must always use the main merchant's API secret.

Callback Body (Success JSON)

Callback Body (Success JSON)

```
{
  "success": true,
  "status": "SUCCESS",
  "statusCode": "200",
  "type": "payout",
  "mode": "P2P",
  "orderId": "PO-98765",
  "paymentId": null,
  "payoutId": 4412,
  "amount": 1000.00,
  "currency": "BDT",
  "transactionId": "out_111",
  "reference": "out_111",
  "accountNumber": "01711111111",
  "accountTitle": null,
  "rail": "MW",
  "reason": "SUCCESS"
}
```

Callback Body (Failed JSON)

Callback Body (Failed JSON)

```
{
  "success": false,
  "status": "FAILED",
  "statusCode": "400",
  "type": "payout",
  "mode": "P2P",
  "orderId": "PO-98765",
  "paymentId": null,
  "payoutId": 4412,
  "amount": 1000.00,
  "currency": "BDT",
  "transactionId": "out_111",
  "reference": "out_111",
  "accountNumber": "01711111111",
  "accountTitle": null,
  "rail": "MW",
  "reason": "FAILED"
}
```

Callback Body (Encrypted)

Callback Body (Encrypted)

```
{
  "encrypted": true,
  "alg": "AES/GCM",
  "iv": "NIIdWfHS6pNe9ow7e",
  "cipherText": "Wquopg9kMpJpkCbGi3Li72EwMB1..."
}
```

Status Codes

Reference for response status codes



200

Status Code

Success



202
Status Code

Pending



400
Status Code

Failed



404
Status Code

Unknown